When Does Cross-Adapter KV Cache Sharing Pay Off?

A Study of Multi-Role Coding Agents

RoleKV

October 18, 2026

Abstract

RoleKV uses activated LoRA (aLoRA) to share key-value (KV) caches between a Coder and a Reviewer, rebuilding each role's adapted states at handoff. We evaluate six methods using Qwen2.5-Coder-1.5B-Instruct. On 100 MBPP tasks with test-suite feedback, this H1 policy reduces mean processed prefill tokens by 68.7% and median switch time to first token from 139 to 45 ms relative to per-role LoRA with full recomputation. H1 passes 61 tasks versus 59, without establishing quality superiority or equivalence. Fixed four-turn replays isolate the systems effect: H1 processes 3.6–3.9× fewer prefill tokens as context grows from 1.1k to 8.1k, while summed model-turn time changes from 4% slower to 13% faster. Shared-adapter and mismatched-cache baselines remain competitive. Among H1's 41 second-round approvals, 35 programs fail the suite. The result is a measured gain in prefill volume and switch responsiveness, with a review failure that limits the interpretation of closed-loop time savings.

1 Introduction

In a Coder–Reviewer loop, one role writes a program and another inspects the code and execution feedback before requesting a revision. Execution-guided refinement can improve generated programs [1], but every attempt adds history for both roles to read. LoRA adapters specialize these roles on a shared backbone [2]. Sharing weights does not guarantee compatible key-value (KV) states: the same text can produce different caches under different adapters [3]. Our full-recomputation baseline therefore reprocesses the accumulated history under the incoming role's adapter.

We build RoleKV on activated LoRA (aLoRA), an existing mechanism that leaves states before an invocation unchanged [3]. Each role's adapter activates at its header. At the next handoff, our H1 policy rebuilds the previous role's adapted span with base weights, preserving one shared base-model prefix (Figure 1). H2 skips that reconstruction and keeps mixed-adapter states. The engineering challenge is to preserve activation and cache semantics through decoding while accounting for the work saved and the work added at handoff.

Our question is *when does cross-adapter KV sharing reduce model cost without compromising coding-task quality?* A long common prefix creates more repeated prefill, while decoding and handoff reconstruction can offset the saving. A premature approval can also make a session cheaper by stopping it early. We separate these effects through three contributions:

- **An instrumented runtime:** six cache/adapter policies, explicit aLoRA activation through decoding, and logged token accounting, including handoff reconstruction, used by a replay dashboard.
- **Two complementary evaluations:** fixed four-turn traces across four context lengths, and 600 closed-loop sessions on 100 paired MBPP tasks [4] using five trained adapters.
- **Benefits and failure modes:** large prefill and switch-latency gains coexist with modest context-dependent model-time gains, competitive simpler baselines, and a second-round review failure.

Our contribution is the implementation and empirical study of these trade-offs. The underlying aLoRA sharing mechanism comes from prior work.

2 Background and Related Work

2.1 Code Repair and Shared History

Iterative code repair revisits a task together with earlier programs and their execution results. Self-Debugging uses generation, explanation, feedback, and revision to improve code with an existing language model [1]. Its MBPP evaluation exposes one test and reserves two for final scoring: execution feedback can reveal a failure, while passing the visible test leaves further errors undetected. This motivates a Coder–Reviewer loop in which both roles inspect a growing common history. The benefit of separate role adapters remains an empirical question, since a single model can also perform generation and review.

2.2 LoRA and KV Compatibility

LoRA specializes a frozen backbone through trainable low-rank weight updates [2]. S-LoRA makes many such adapters practical to serve through heterogeneous batching and unified memory management for adapter weights and KV tensors [5]. These optimizations exploit shared backbone weights; cross-adapter KV reuse additionally requires compatible activations. An adapter can change cached keys and values directly through projection updates or indirectly through hidden states produced by earlier layers [3].

Ordinary autoregressive caching retains earlier keys and values during decoding. vLLM's PagedAttention manages these tensors in blocks and supports sharing within and across

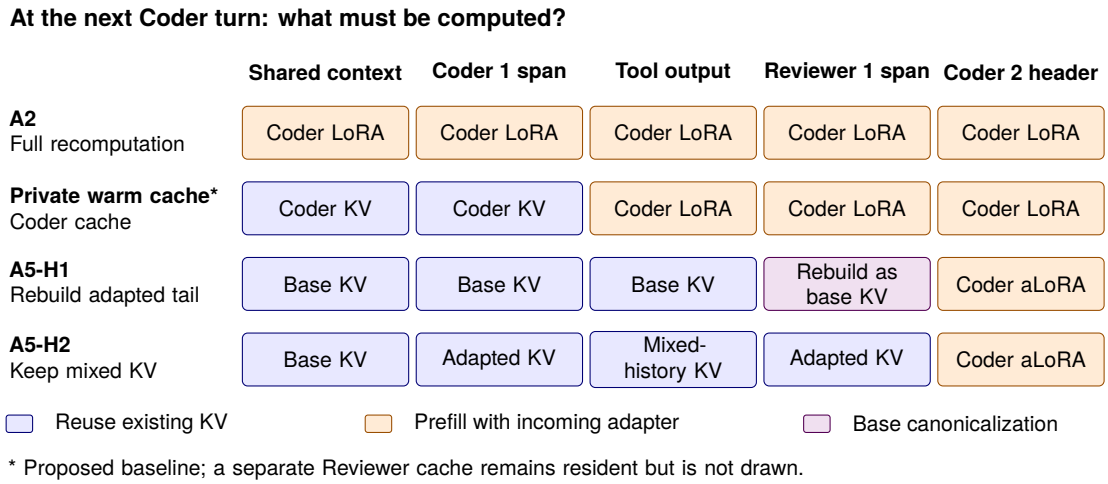


Figure 1: Cache preparation on returning to the Coder. Blue spans reuse existing states; orange spans require incoming-role prefill; purple spans are rebuilt with base weights. H1 has already canonicalized Coder 1 at the preceding handoff and now rebuilds Reviewer 1. Equal-width spans are schematic, not token counts; boundary tokens are omitted. The private warm cache policy is conceptual and not evaluated. H2's tool states are also conditioned on the retained adapted history.

requests [6]. With an unchanged model and compatible positions, identical prefixes can reuse their states. Identical text under different adapters does not generally satisfy that condition. The resulting trade-off depends on cache policy: full recomputation uses one live cache, whereas private warm caches retain a prefix for each role and compute its missing history on return. Cross-adapter sharing must therefore be assessed against both recomputation cost and the memory required to keep private caches resident.

2.3 Exact Cross-Adapter Sharing

Activated LoRA (aLoRA) preserves base-model states before an invocation sequence and applies adaptation from that sequence onward [3]. This enables exact reuse of the base-processed prefix. Li et al. integrate the mechanism into vLLM through base-aligned block hashing and activation-aware masking [7]. The boundary is important for sequential role invocations: the aLoRA paper notes that adapter-generated states cannot be directly reused by another adapter or the base model, so the generated text must be prefilled again. Our H1 policy applies this recomputation at handoff, including the adapted role header.

ICaRus takes a different approach: it freezes a shared logical encoder that writes KV states and trains task-specific logical decoders that read them [8]. During generation, the base and adapted paths run together, with shared weight reads and combined attention queries reducing their execution overhead. Thus, ICaRus maintains reusable states throughout decoding, whereas H1 reconstructs them in a contiguous pass at handoff. ICaRus evaluates code-generation quality and HotpotQA-based serving workloads; those experiments do not directly measure our code-repair loop. In both architectures, exact cache reuse concerns states under the chosen adaptation design; task quality relative to conventional LoRA requires separate evaluation.

2.4 Approximate Sharing and Reconstruction

Other approaches reuse representations across model variants while managing the resulting mismatch. DroidSpeak selectively recomputes layers of another variant's cache and reuses the rest [9]. LRAgent instead decomposes the value cache into a shared base component and a compact adapter-dependent component [10]. BaseShared maintains per-agent low-rank caches for standard adapters; BaseLRShared further shares the low-rank cache using a shared-A adapter architecture. Sharing the base weights alone does not make the base component identical, because its input hidden states can still depend on the producing adapter.

ForkKV combines disaggregated shared and agent-specific caches with a DualRadixTree and a reconstruction kernel for multi-LoRA serving [11]. PReCache precomputes each adapter's low-rank cache; its ReBaseShared variant additionally constructs a neutral base cache from adapter-free hidden states [12]. The lazy-prefill schedule performs this reconstruction after an agent's turn. This resembles H1 in scheduling base-state reconstruction at a handoff, but serves a different purpose: PReCache approximates standard adapters' representations, whereas H1 restores the base prefix expected by the incoming aLoRA. These distinctions separate the choice of reusable representation from the cost of maintaining it.

2.5 Scope and Measurement

Prior work establishes exact and approximate paths to cross-adapter sharing. We evaluate aLoRA handoffs in a small-model Coder–Reviewer loop, including the cost of reconstruction and decoding. Fixed transcripts isolate systems cost, while closed-loop runs reveal changes in generated code, review decisions, and session length. PReCache also distinguishes controlled efficiency traces from actual trajectories [12]. Our baselines test full recomputation, one shared adapter, and mismatched reuse; the other systems reviewed

here provide context rather than direct experimental comparisons.

3 Research Questions

1. How does shared-context length affect prefill volume, switch latency, and model-turn time when the conversation is fixed?

2. How do cache handoff policies affect task quality and model cost?

3. Do per-role adapters and exact sharing offer a measured advantage over one shared adapter or mismatched-cache reuse in this workload?

4 Methods and Experimental Design

We use two complementary experiments: fixed-output traces that hold behavior constant, and closed-loop evaluation in which each method generates its own trajectory. Training records, event logs, and their hashes are indexed in `report/methods_manifest.json` and `report/results_summary.json`. All primary results use an RTX 4090 Laptop GPU with 16 GB under WSL2, bf16 inference, greedy decoding, and the same model snapshot `2e1fd397`. The recorded environment uses PyTorch 2.14.1+cu126, Transformers 5.20.1, and PEFT 0.21.2. An additional 4070 base-model run is excluded from the primary comparison.

The project repository includes the runtime, training and evaluation scripts, recorded runs, and a dashboard that replays the event logs. The offline report script verifies paired task sets and accounting totals and regenerates the result tables and plots without invoking the model.

4.1 Tasks and Shared Context

The evaluated model is Qwen2.5-Coder-1.5B-Instruct [13]; the 0.5B variant is used for development. The task loader uses MBPP's `full` configuration [4], pooling the train, validation, and prompt splits for data generation and selecting 100 test-split tasks with Python's random sampler at seed 0. It checks that the training and selected evaluation task IDs are disjoint.

Each TASK segment contains a repository-style `utils.py`, the task description, and its first assert. The context builder samples 30 reference solutions from the pooled training tasks by default, excluding the target task; selection is deterministic from a hash of the task ID. These functions provide shared text to process, not evidence that the task requires a real repository. In the current SFT exports, TASK segments range from 1,276 to 2,497 development-tokenizer tokens, with a median of approximately 1,809. For the 100 evaluated tasks, logged TASK lengths range from 1,280 to 2,710 tokens, with a median of 1,804. The added context and evaluation subset are study-specific augmentations.

4.2 Teacher Trajectories and SFT Examples

The generator requests a solution or an intentionally faulty first attempt from a teacher, executes the code, and retains a trajectory only if its final code passes the full assert suite. A requested faulty attempt must fail that suite; its feedback is generated from the real test output, followed by a tested repair. Passing attempts receive a deterministic `APPROVE` target. The configured teacher identifier is `claude-sonnet-5-5`; the export does not record the API model revision or generation arguments, so these remain provenance information to recover from the generation run.

The current export contains 280 accepted trajectories from distinct tasks: 222 Coder–Reviewer sequences and 58 Coder–Reviewer–Coder repair sequences. They yield 338 Coder and 280 Reviewer SFT rows, with 222 `APPROVE` and 58 `REVISE` labels. The Coder file retains the 58 faulty initial attempts as targets as well as their repairs; it represents trajectory imitation, not exclusively correct-solution supervision. No Reviewer turn after a successful repair is included in these three-turn trajectories. The accepted distribution therefore differs from the generator's nominal 50% bug-request default.

Prompts use separately tokenized ChatML segments. The system segment is shared with inference, and a role's complete assistant header is its aLoRA invocation sequence. Labels mask all context tokens with -100 and supervise only the target BODY, including its closing delimiter. The two files' median lengths are 2,036 and 2,060.5 tokens for Coder and Reviewer, respectively; the maximum is 2,932. These lengths were checked locally using the development tokenizer. Full data hashes appear in the manifest and `data/README.md`.

4.3 Adapter Training

We train role-specific standard LoRA and aLoRA adapters, plus a shared LoRA on both roles for A1. Table 1 records the settings in the saved training runs. Only the seven attention and MLP linear projections are targeted; embeddings and the language-model head are excluded to preserve the base-prefix sharing assumptions. Loss is computed only on the role's target tokens.

REVISE oversampling repeats those Reviewer examples in the training input, including when training the shared adapter; it leaves the exported data files unchanged. Each Coder adapter uses 338 rows per epoch and 85 optimizer updates over two epochs. Each Reviewer uses 396 rows after oversampling and 99 updates; the shared adapter uses 734 rows and 184 updates. Loss averages target tokens within each row and then averages rows in an accumulation window, without sequence packing. The training records reject no input rows, report zero reload error for all five adapters, and record zero pre-header logit difference for both aLoRAs on their saved probe. The records identify training commit `79e1f6a`; weights are loaded from `adapters/1.5b`. Each aLoRA contains 36.9 million trainable parameters, versus 18.5 million for each standard LoRA.

4.4 Session Protocol and Test Feedback

The runtime follows Coder $\rightarrow$ tool output $\rightarrow$ Reviewer. It uses greedy decoding with default limits of 512 new Coder tokens and 128 Reviewer tokens, stopping on ChatML end-of-turn or end-of-text tokens. An `APPROVE` or `INVALID` Reviewer verdict ends the session. After the second `REVISE`, the next Coder output is final, without a further tool call or review, for at most five role turns. Tests execute in a temporary-directory Python

Setting	Recorded value
Standard LoRA / aLoRA rank	16 / 32
Scaling α	$2r$
Targets	q, k, v, o, gate, up, down projections
Learning rate / epochs	2×10^{-4} / 2
Maximum sequence length	4,096 tokens
Training precision	bf16 base / fp32 adapters
Inference precision	bf16 base and adapters
Dropout / weight decay	0.05 / 0
Optimizer	AdamW, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$
Microbatch / accumulation	1 row / 8 rows
Schedule	5% linear warm-up, then linear decay
Seed / packing / clipping	0 / none / none
Checkpointing	Non-reentrant
Reviewer REVISE multiplicity	3×

Table 1: Training settings verified from all five saved run records. LoRA and aLoRA ranks differ, so the comparison is not parameter-matched.

subprocess with a five-second timeout and an output cap of 2,000 characters.

Feedback boundary. Although only the first assert appears in the TASK text, the current data generator and default evaluation runner execute *all* MBPP asserts in the tool call and return the resulting stdout and stderr to the model. Final code is scored against the same suite. These are test-suite-assisted repair results, not evaluation on tests withheld throughout the interaction; failure output can reveal additional assertions. A runner option restricts feedback to the first assert, matching the visibility boundary used by Self-Debugging [1], but that is a separate protocol whose runs must be labeled and analyzed separately. Comparisons must hold the feedback policy fixed, and equal feedback access does not establish generalization to unseen tests.

4.5 Cache Policies and Correctness

The implemented runtime uses one logical token history and one `DynamicCache` per session, with the following policies:

- **A2 (full recomputation):** discard the cache at each turn and prefill the entire history and new role header under the incoming role's standard LoRA.
- **A5-H1 (canonicalize):** retain the base-processed prefix, crop the previous role's adapted span, and rebuild it with base weights at the start of the next turn. This covers the previous role's header and output. New tool and boundary tokens are processed in the same forward; the incoming aLoRA activates only at its own header. The final session turn is never canonicalized because there is no subsequent handoff.
- **A5-H2 (keep):** retain the previous role's adapted KV, process new input, and activate the incoming aLoRA at its header. The resulting history contains states produced by different adapters.

All aLoRA decode steps receive explicit activation offsets so that adaptation remains active when the input chunk no longer contains the invocation header. A0 uses the base model, A1 uses one shared standard LoRA, and A3 retains mixed-adapter history with standard role LoRAs; these policies also have runtime implementations. H1 and H2 use the same aLoRA weights. A0 and A1 retain their compatible single-model cache; A3 switches standard role LoRAs without rebuilding the previous role's states.

Inference uses PEFT 0.21.2 with scaled dot-product attention. A contiguous-slice aLoRA path and persistent activation-offset hooks replace masked row selection and per-call hook registration. Timings characterize this optimized runtime; correctness checks cover activation behavior, cache restoration, positions, and token accounting.

Private warm caches remain unimplemented in the inspected runtime. The proposed policy keeps a valid prefix per standard role adapter and computes missing history on return. Its comparison with A2 must use identical adapters and count all resident caches. Figure 1 illustrates policies independently of checkpoint quality.

The test suite compares cached logits with a full forward under the *same aLoRA*, and canonicalized KV with a fresh base-model forward. These tests use the 0.5B development model and nonzero random rank-8 adapters. The recorded pre-evaluation gate passed 60/60 tests in fp32 on CUDA. Four bf16 equivalence comparisons failed their tolerances in the development tests, as documented in the run notes. Consequently, exact sharing describes H1's representation invariant, not bitwise equality of all bf16 schedules or completed equivalence validation of the trained 1.5B deployment. Ordinary LoRA is a task-quality reference rather than H1's equivalence target. H2 retains mixed states and intentionally does not satisfy the H1 invariant.

4.6 Fixed-Trace System Measurements

The cost sweep replays one four-turn conversation: faulty Coder output, `REVISE`, repaired Coder output, and `APPROVE`, with fixed tool text. It generates 197 tokens per session, including stop tokens, and makes three role switches. We vary TASK length across 1,101, 2,100, 4,095, and 8,114 tokens, comparing A2, H1, and H2 over five repeats each. Methods are interleaved after warm-up in a fixed order. The sweep uses untrained adapters at the measured ranks and precision, so it measures execution cost rather than learned behavior. Its forced `passed` flag denotes replay completion and is excluded from quality analysis. Output length and switch count are fixed; their independent effects are not evaluated.

Processed prefill tokens sum new prefill and canonicalization tokens, counting every non-decoding token position fed to the model. This is a token-volume metric rather than measured FLOPs. Switch TTFT is the median over every turn after turn zero, including H1 reconstruction. Model-turn time

sums synchronized turn durations, including decoding and host overhead inside each turn, but excludes tool execution, final scoring, and model loading. We report median session time and its observed range across the five repeats. The logged `peak_kv_mb` is the maximum logical KV tensor size at turn boundaries, not peak device allocation; we make no measured memory-saving claim.

4.7 Quality and Reviewer Behavior

Each of the six methods generates one greedy trajectory for each of the same 100 tasks. The log contains all 600 final task outcomes and 1,923 model turns. We report final-code full-suite pass counts, mean turn counts, generated token counts, and invalid or truncated outputs, retaining stopped and failed sessions. The code may undergo up to two revisions, so the final task pass rate is a session-level success measure rather than single-attempt Coder accuracy; we avoid interpreting the source summaries' `pass@1` label as unrevised code-generation performance.

For second-round reviews, we count approvals and the fraction whose final code fails the suite. An approval terminates the session without changing the reviewed code, so the final outcome scores that approval directly. This denominator includes all second-round approvals, without conditioning on first-assert success. No second review occurs in the SFT trajectories. Initial accuracy, repair, regression, and visible-test-conditioned approval rates require scoring intermediate code, whose outcomes are not stored in the event log; they are outside this analysis.

4.8 Cost–Quality Interpretation

Closed-loop methods take different trajectories, so fewer processed tokens can reflect premature approval or failure rather than a better cache policy. We interpret cost jointly with task success and Reviewer behavior, using a paired A2–H1 success comparison. We report Wilson 95% intervals for their individual success proportions and an exact two-sided McNemar test on discordant tasks. These quantify task-level sampling uncertainty; they do not measure variation across training seeds or decoding runs. A small observed gap does not establish equivalence. Tool-inclusive task wall times are not stored, so every reported time comparison concerns model turns.

5 Results and Analysis

5.1 Fixed Traces: Savings Depend on Context

Figure 2 separates prefill volume, role-switch latency, and summed model-turn time while holding generated text constant. H1 processes 1,576 versus A2's 5,599 prefill tokens at 1.1k context, and 8,589 versus 33,651 at 8.1k: reductions of 3.55× and 3.92×, respectively. Every H1 session includes 221 canonicalization tokens. H2 avoids those 221 tokens, but retains adapted states rather than restoring the base prefix. The growing ratio follows from amortizing the shared prefix over four turns; it does not imply that all model computation is reduced by the same factor.

A2's median switch TTFT rises from 94 to 583 ms, whereas H1 stays between 47 and 50 ms across the four contexts. The resulting latency ratio grows from 1.89× to 11.71×. This is the clearest measured benefit of sharing: the incoming role avoids reprocessing a growing common prefix.

The model-time benefit is substantially smaller. At 1.1k, H1 takes 6.84 s versus 6.56 s for A2, about 4% slower. At 2.1k the medians are 6.89 and 6.82 s. H1 becomes slightly faster at 4.1k (7.01 versus 7.19 s), and is 13% faster at 8.1k (7.32 versus 8.40 s). At that longest context, observed session ranges are 7.16–7.55 s for H1 and 8.36–8.46 s for A2. Median decode time is approximately 34.2–34.4 ms per token for H1, compared with 31.5–31.9 ms for A2. Residual aLoRA execution overhead therefore offsets some prefill savings. The crossover is specific to this trace, hardware, adapter configuration, and implementation; it is not a universal context threshold.

5.2 Closed-Loop Quality and Model Cost

Table 2 reports all six methods on the same 100 tasks. H1 passes 61 tasks and A2 passes 59. Paired by task, both pass 51, only A2 passes 8, only H1 passes 10, and neither passes 31. The exact two-sided McNemar p-value is 0.815. Individual Wilson 95% intervals are 49.2–68.1% for A2 and 51.2–70.0% for H1. These results provide no detected success-rate difference on this sample, but do not establish equivalence or non-inferiority.

H1 averages 2,197 processed prefill tokens per task, including 138 canonicalization tokens, versus 7,013 for A2: 68.7% fewer. Median switch TTFT falls from 139.4 to 45.4 ms, a 3.07× ratio. Mean model-turn time falls from 5.56 to 4.29 s, or 22.8%. This last reduction combines cache behavior with different generated trajectories. H1 averages 2.94 turns and 124.8 generated tokens, compared with A2's 3.27 turns and 170.2 tokens. It therefore cannot be attributed wholly to caching. The fixed replay provides the cleaner systems comparison.

The other baselines also constrain the interpretation. A1, which uses one shared standard LoRA and a compatible cache, passes 61 tasks with 2,060 prefill tokens per task. A3, which retains mismatched per-role LoRA states, passes 62 with 2,061 prefill tokens. H2 passes 60, with lower prefill volume than H1 but the same second-round review weakness discussed below. Thus, this workload does not demonstrate a quality advantage for per-role adapters over a shared adapter, or for exact over mismatched cache reuse. The mechanistic distinction between H1 and A3 remains important, but it does not substitute for an observed task-quality advantage.

5.3 Second-Round Review Failure

Table 3 reveals a failure hidden by aggregate task pass rates. H1 approves 41 of 45 second reviews; 35 of those 41 approvals (85.4%) concern code that fails the suite. H2 approves 39 of 45, including 34 failures (87.2%). A2 approves only 2 of 43 second reviews, and neither approved program fails; A1 and A3 likewise have no failing second-round approvals in these runs. Zero observed errors on their small approval denominators should not be read as guaranteed reviewer reliability.

The SFT trajectories never include a review after a repair, making this stage outside the training distribution. Both aLoRA policies show the failure, so canonicalization alone is

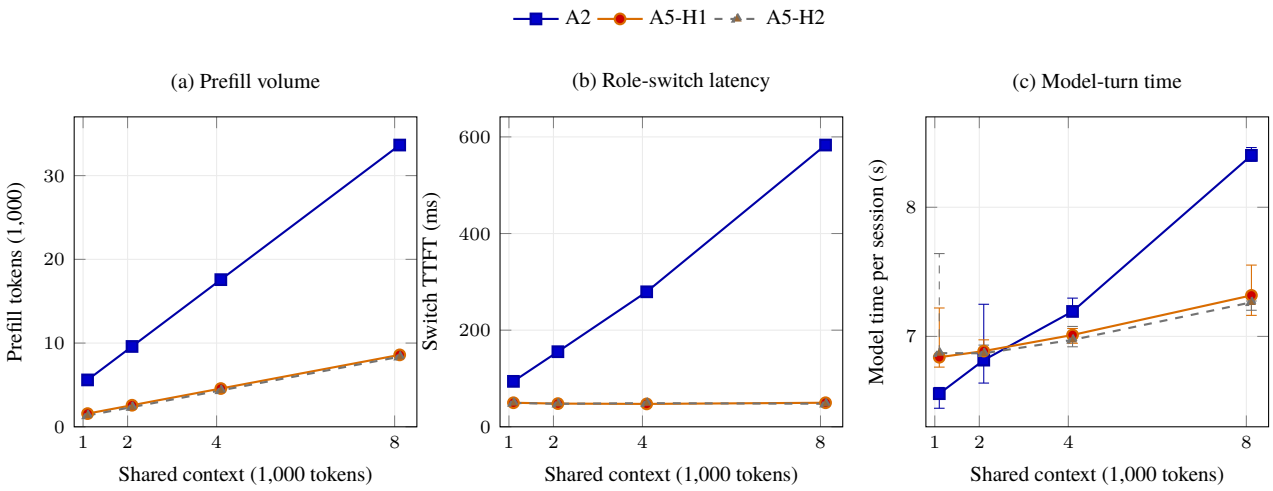


Figure 2: Fixed four-turn replay on the RTX 4090 Laptop GPU. All methods generate the same 197 tokens and make three role switches, using untrained adapters. Prefill volume includes canonicalization. TTFT is the median over the 15 post-initial turns across five repeats. Model time is the median sum of turn durations per session; whiskers show the minimum and maximum of five sessions, not confidence intervals. Panel (c) uses a truncated vertical axis to show the crossover.

Method	Pass / 100	Prefill tokens	Canon tokens	Switch TTFT (ms)	Model time (s)	Turns	Generated tokens
A0	51	2,038	0	19.9	4.29	3.58	268.9
A1	61	2,060	0	35.3	5.29	3.24	175.3
A2	59	7,013	0	139.4	5.56	3.27	170.2
A3	62	2,061	0	40.9	5.54	3.24	182.5
A5-H1	61	2,197	138	45.4	4.29	2.94	124.8
A5-H2	60	2,059	0	43.7	4.40	2.96	133.8

Table 2: Closed-loop results on 100 paired MBPP tasks per method, all on the 4090 with test-suite feedback. Pass counts score the final code after up to two revisions. Prefill, canonicalization, model time, turns, and generated tokens are means per task; canonicalization is included in prefill. TTFT is the pooled median over turns after turn zero. Model time excludes tools, final scoring, and loading. Generated tokens include stop tokens.

Method	Reviews	Approvals	Failed / approved
A0	56	0	–
A1	42	2	0 / 2
A2	43	2	0 / 2
A3	43	5	0 / 5
A5-H1	45	41	35 / 41
A5-H2	45	39	34 / 39

Table 3: Second-round Reviewer decisions. "Failed / approved" counts suite-failing final programs among second-round approvals. A dash denotes no approvals. These are conditional counts for the tasks that reached this round, not matched subsets across methods.

unlikely to explain it. Training coverage is a plausible contributor, but establishing a cause requires controlled retraining or other ablations. Because `APPROVE` stops the session, premature approval also removes a possible final repair turn. The final-code scoring remains valid for the observed session, but the results do not show how many of those failures another repair would fix.

Case study: MBPP task 135. The H1 Coder repeats the same return expression for `hexagonal_num` after a revision request, yet the second Reviewer approves it. The expressions and verdicts below come from the recorded turns:

Coder 1	`n * (2 * n * n - 1)`
Reviewer 1	`REVISE`
Coder 2	`n * (2 * n * n - 1)`
Reviewer 2	`APPROVE`

The final program fails the suite. A2 returns $n(2n-1)$ and passes after its first review. H1 processes 2,212 prefill tokens including 133 reconstruction tokens, versus A2's 3,818, but its cheaper session ends in failure. This selected example illustrates a false approval after unchanged code; it does not estimate the frequency of such failures.

Format errors differ from approval errors. Every adapted Reviewer emits a parseable verdict, despite 15–46 truncated Reviewer turns per method. A0 emits 54 `INVALID` verdicts among 156 reviews and never approves; it truncates 90 Reviewer and three Coder turns. A verdict parsed from the first line can remain valid when its explanation reaches the token

cap. Session behavior therefore needs to accompany success and cost.

6 Limitations and Discussion

6.1 What the Experiments Establish

The answer depends on what is held fixed. With identical generated text, longer shared context increases H1's advantage in switch latency, while decode overhead limits total model-time savings. With generated trajectories, similar final pass counts conceal different review decisions and opportunities for repair. Finally, A1 and A3 remain competitive, so per-role specialization and exact sharing have no demonstrated quality premium in this benchmark.

H1's base-prefix invariant provides a clear cache contract for independently trained role adapters. Its engineering value and the measured prefill savings stand alongside the quality and numerical-validation limits described above; they do not establish that H1 is the best quality–cost policy overall.

6.2 Coverage and Measurement Limits

The study uses one 1.5B backbone, two roles, one training seed, and one greedy run per method on 100 function-level tasks. The added reference functions create shared context but do not turn MBPP into repository-level software engineering. Full-suite tool feedback further limits comparison with protocols that withhold tests. Different ranks and parameter counts also prevent attributing quality differences solely to cache policy when comparing standard LoRA with aLoRA.

The fixed sweep uses one synthetic transcript and untrained adapters. Five timing repeats describe local variation, not run-to-run variation across machines or sessions with different text. The closed-loop runs execute methods in separate blocks rather than repeated randomized order. All primary timings are from one laptop GPU under WSL2. The separate 4070 A0 result is 54/100 rather than the 4090's 51/100, reinforcing the need to keep the main comparison on one machine. We do not report tool-inclusive wall time, measured peak GPU memory savings, or an energy reduction.

Private warm caches are illustrated but not implemented or timed. Output length, role count, and switch frequency are not independently swept. Exact sharing may lose its time advantage when role outputs become longer, because reconstruction grows with the adapted span, but this dependence has not been measured here. The current evidence cannot locate a general break-even boundary against a memory-resident private cache.

6.3 Next Experiments

The first priority is to add multi-round review trajectories and repeat evaluation with several training seeds, while separately scoring initial code, repairs, and regressions. A matched-weight full-recomputation aLoRA baseline at the deployed precision would isolate H1's cache handling from adapter training and strengthen numerical validation. A factorial trace sweep over output length and switch count, an implemented private-cache baseline, and tool-inclusive timing would then test the remaining cost questions. Evaluations with tests withheld from feedback and larger, repository-level workloads are needed before generalizing the quality findings beyond this setup.

7 Conclusion

Cross-adapter KV sharing reduces repeated prefill and role-switch latency in the RoleKV Coder–Reviewer runtime. On fixed four-turn traces, H1's switch latency stays near 50 ms while full recomputation grows to 583 ms at 8.1k context; model-time savings emerge only as the shared prefix becomes large enough to offset decoding overhead. On 100 closed-loop tasks, H1 passes 61 versus A2's 59, with no detected paired success-rate difference, but frequent false approvals in second reviews weaken the interpretation of its lower task cost. Shared-adapter and mismatched-cache baselines are also competitive. The practical lesson is to measure reconstruction, decoding, and review behavior together: a lower prefill count is most useful when it reduces a substantial part of the session's work, and a lower task cost must be interpreted alongside how the session ended.

References

[1] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. arXiv preprint arXiv:2304.05128, 2023. https://arxiv.org/abs/2304.05128.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. https://arxiv.org/abs/2106.09685.

[3] Kristjan Greenewald, Luis Lastras, Thomas Parnell, Vraj Shah, Lucian Popa, Giulio Zizzo, Chulaka Gunasekara, Ambrish Rawat, and David Cox. Activated LoRA: Fine-tuned LLMs for intrinsics. arXiv preprint arXiv:2504.12397, 2025. Version 5. https://arxiv.org/abs/2504.12397v5.

[4] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021. https://arxiv.org/abs/2108.07732.

[5] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. In *Proceedings of Machine Learning and Systems*, 2024. https://arxiv.org/abs/2311.03285v3.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. https://arxiv.org/abs/2309.06180.

[7] Allison Li, Kristjan Greenewald, Thomas Parnell, and Navid Azizan. Efficient multi-adapter LLM serving via cross-model KV-cache reuse with activated LoRA. arXiv preprint arXiv:2512.17910, 2025. https://arxiv.org/abs/2512.17910.

[8] Sunghyeon Woo, Jaeeun Kil, Hoseung Kim, Minsub Kim, Joonghoon Kim, Ahreum Seo, Sungjae Lee, Minjung Jo, Jiwon Ryu, Baeseong Park, Se Jung Kwon, and Dongsoo Lee. ICaRus: Identical cache reuse for efficient multi model inference. arXiv preprint arXiv:2603.13281, 2026. https://arxiv.org/abs/2603.13281v1.

[9] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. DroidSpeak: KV cache sharing across fine-tuned model variants. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 319–338. USENIX Association, 2026. https://www.usenix.org/conference/nsdi26/presentation/liu-yuhan.

[10] Hyesung Jeon, Hyeongju Ha, and Jae-Joon Kim. LRAgent: Efficient KV cache sharing for multi-LoRA LLM agents. In *International Conference on Machine Learning*, 2026. https://arxiv.org/abs/2602.01053v2.

[11] Shao Wang, Rui Ren, and Lin Gui. ForkKV: Scaling multi-LoRA agent serving via copy-on-write disaggregated KV cache. arXiv preprint arXiv:2604.06370, 2026. https://arxiv.org/abs/2604.06370.

[12] Hyesung Jeon, Hyeongju Ha, and Jae-Joon Kim. PReCache: Efficient KV cache sharing for multi-LoRA agents via low-rank precomputation and neutral reconstruction. arXiv preprint arXiv:2609.34054, 2026. Version 2. https://arxiv.org/abs/2609.34054v2.

[13] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder technical report. arXiv preprint arXiv:2409.12186, 2024. Version 3. https://arxiv.org/abs/2409.12186v3.